Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

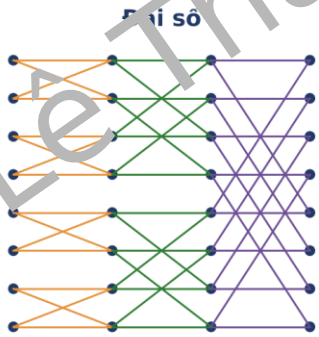
TOÁN HỌC TRONG TIN HỌC

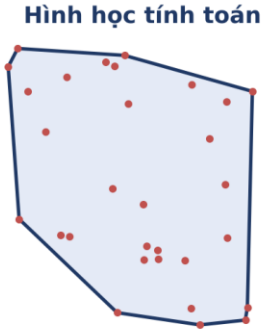
Chuyên đề nâng cao ôn thi học sinh giỏi Quốc gia (VOI)

TẬP 1 · LÝ THUYẾT









Lê Thành Nam

Chương 1. SỐ HỌC

Chương 2. TỔ HỢP – XÁC SUẤT – TRÒ CHƠI

Chương 3. ĐẠI SỐ

Chương 4. HÌNH HỌC TÍNH TOÁN

LỜI NÓI ĐẦU

Bộ tài liệu này dành cho học sinh đã vững phần toán của đề thi cấp tỉnh: ghép cặp ước tới √n, sàng Eratosthenes, thuật toán Euclid, lũy thừa nhanh, tổ hợp cơ bản, tổng tiền tố, tích có hướng. Ở kì thi chọn học sinh giỏi Quốc gia (VOI), những công cụ đó vẫn xuất hiện, nhưng hiếm khi một mình: chúng được ghép với nhau, được đẩy tới giới hạn dữ liệu lớn hơn hàng nghìn lần, và thường ẩn sau một câu chuyện mà người giải phải tự nhận ra mô hình toán học.

Vì vậy, mỗi bài học trong tập này đều xuất phát từ một bài toán có giới hạn kiểu VOI (n tới 10^7, 10^{12} hay 10^{18}; hàng trăm nghìn truy vấn). Ta bắt đầu bằng cách làm mà một học sinh cấp tỉnh giỏi sẽ nghĩ ra, đếm chính xác số phép tính để thấy nó dừng ở subtask nào, rồi dựng công cụ mới từng bước một. Mỗi công cụ đều có phần **vì sao đúng**, vì ở mức này, hiểu chứng minh là cách duy nhất để biến đổi công cụ cho hợp với bài toán mới.

Bộ sách gồm ba tập đi cùng nhau:

• **Tập 1: Lý thuyết** (cuốn sách này) gồm 15 bài học chia thành 4 chương: Số học; Tổ hợp – Xác suất – Trò chơi; Đại số; Hình học tính toán.

• **Tập 2: Bài tập** gồm 45 bài hoàn toàn mới, mỗi bài học có ba bài ở ba mức (★, ★★, ★★★ theo mặt bằng VOI), có subtask như đề thi thật.

• **Tập 3: Hướng dẫn giải** phân tích từng bài từ cách ngây thơ tới cách tối ưu, kèm chương trình và bảng điểm đo thật của từng cách làm.

Mọi chương trình trong sách đều đã được biên dịch theo chuẩn C++14, chạy thử, và đối chiếu với lời giải vét cạn trên nhiều bộ dữ liệu ngẫu nhiên. Mọi con số trong các bảng chạy tay đều được kiểm tra lại bằng máy. Chương trình đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được với cả Themis lẫn các trang chấm trực tuyến.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, tính được bằng tay, chứa sẵn mầm của ý tưởng chính.
Bài toán	Đề bài cụ thể với giới hạn kiểu VOI, có ví dụ.
Ý tưởng ngây thơ	Cách làm của một học sinh giỏi cấp tỉnh, kèm phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác chỗ cách làm hiện tại không vượt qua được.
Phá bế tắc	Từng bước: quan sát, công thức, chứng minh, chạy tay, chương trình.
Mở rộng	Biến thể và các kĩ thuật đi kèm thường gặp trong đề VOI.
Bẫy thường gặp	Những lỗi làm mất điểm ở mức này: tràn số, sai trường hợp biên, quá bộ nhớ.
Ghi nhớ	Tóm tắt cốt lõi.
Luyện tập	Liên hệ tới ba bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
CÔNG THỨC	Công thức, định lí hoặc thuật toán cốt lõi.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và bộ nhớ.
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Lê Thành Nam

Quy ước kí hiệu. $a \mid b$ nghĩa là a chia hết b (a là ước của b). $\lfloor x \rfloor$ là phần nguyên dưới của x, $\lceil x \rceil$ là phần nguyên trên. gcd và lcm là ước chung lớn nhất và bội chung nhỏ nhất. $a \equiv b \pmod m$ nghĩa là $m \mid a - b$. Phép XOR theo bit được viết là $\oplus$ trong lời văn và `^` trong chương trình. Tổng $\Sigma_{d \mid n} f(d)$ là tổng của $f(d)$ trên mọi ước dương d của n. Khi đề yêu cầu in một số hữu tỉ P/Q "theo modulo p", ta in $P \cdot Q^{-1} \bmod p$, trong đó Q^{-1} là nghịch đảo của Q theo modulo p.

Bảng tra nhanh: ràng buộc gợi ý độ phức tạp (khoảng 10^8 phép tính đơn giản mỗi giây):

Ràng buộc	Độ phức tạp cần đạt	Ví dụ trong sách
$n \le 5\,000$	$O(n^2)$	Đường đi có vật cản (Bài 5)
$n \le 2 \cdot 10^5$	$O(n \log n)$, $O(n \log^2 n)$	FFT (Bài 12), đường quét (Bài 15)
$n \le 10^7$	$O(n)$	Sàng tuyến tính (Bài 1)
$n \le 10^{12}$	$O(\sqrt{n})$	Chia khối $\lfloor n/i \rfloor$ (Bài 3)
$n \le 10^{18}$	$O(n^{1/4})$, $O(\log n)$	Pollard rho (Bài 4), lũy thừa ma trận (Bài 10)

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. SỐ HỌC **6**

Bài 1. Sàng tuyến tính và hàm nhân tính 7
Bài 2. Euclid mở rộng, phương trình đồng dư và định lí thặng dư Trung Hoa 15
Bài 3. Hàm Möbius và kĩ thuật chia khối ⌊n/i⌋ 23
Bài 4. Miller–Rabin, Pollard rho và căn nguyên thủy 29

CHƯƠNG 2. TỔ HỢP – XÁC SUẤT – TRÒ CHƠI **36**

Bài 5. Định lí Lucas và tổ hợp theo modulo nhỏ 37
Bài 6. Nguyên lí phản xạ, số Catalan và số Stirling 44
Bài 7. Bao hàm – loại trừ nâng cao và bổ đề Burnside 50
Bài 8. Kì vọng và xác suất theo modulo 55
Bài 9. Trò chơi tổ hợp và định lí Sprague–Grundy 60

CHƯƠNG 3. ĐẠI SỐ **65**

Bài 10. Lũy thừa ma trận và truy hồi tuyến tính 66
Bài 11. Khử Gauss, cơ sở XOR và định lí Kirchhoff 72
Bài 12. Biến đổi Fourier nhanh (FFT) và NTT 79

CHƯƠNG 4. HÌNH HỌC TÍNH TOÁN **85**

Bài 13. Bao lồi và kĩ thuật quay đó 86
Bài 14. Bao lồi các đường thẳng và tối ưu quy hoạch động 92
Bài 15. Đường quét 98

Lê Thành Nam

CHƯƠNG 1. SỐ HỌC

*Ở cấp tỉnh, em đã biết duyệt ước tới $\sqrt{n}$, sàng Eratosthenes, Euclid và lũy thừa nhanh. Đề VOI dùng lại đúng những công cụ đó nhưng đặt chúng vào hoàn cảnh khắc nghiệt hơn: cần thông tin số học của **mọi** số tới 10^7 trong thời gian tuyến tính, cần giải phương trình đồng dư với modulo tới 10^{18} mà không tràn số, cần đếm cặp có ước chung cho trước với hàng chục nghìn truy vấn, cần phân tích một số 18 chữ số trong vài mili giây. Chương này trang bị bốn công cụ tương ứng: sàng tuyến tính và hàm nhân tính, Euclid mở rộng cùng định lí thặng dư Trung Hoa, hàm Möbius với kĩ thuật chia khối, và cặp Miller–Rabin – Pollard rho.*

TRONG CHƯƠNG NÀY

Bài 1. Sàng tuyến tính và hàm nhân tính

Bài 2. Euclid mở rộng, phương trình đồng dư, định lí thặng dư Trung Hoa

Bài 3. Hàm Möbius và kĩ thuật chia khối $\lfloor \frac{n}{i} \rfloor$

Bài 4. Miller–Rabin, Pollard rho và căn nguyên thủy

Lê Thành Nam

BÀI 1 Sàng tuyến tính và hàm nhân tính

Học xong bài này, em sẽ:

✓ Biết biến một tổng gcd thành tổng theo ước bằng cách nhóm theo giá trị của gcd.

✓ Hiểu khái niệm hàm nhân tính vì sao chỉ cần biết giá trị của nó tại lũy thừa nguyên tố.

✓ Cài đặt sàng tuyến tính: Mỗi hợp số bị gạch đúng một lần, đồng thời tính φ, μ, d, σ hoặc một hàm nhân tính bất kì cho mọi số tới 10^7 trong O(N).

✓ Ước lượng được bộ nhớ của các mảng sàng và tránh tràn số trong các bước cập nhật.

KHỞI ĐỘNG

Tính tổng gcd(1, 6) + gcd(2, 6) + ... + gcd(6, 6). Viết ra: 1 + 2 + 3 + 2 + 1 + 6 = 15.

Bây giờ hãy nhìn tổng đó theo cách khác: có bao nhiêu số i có gcd(i, 6) = 1? Có bao nhiêu số có gcd bằng 2, bằng 3, bằng 6? Em sẽ thấy các con số 2, 2, 1, 1 xuất hiện, và đó chính là φ(6), φ(3), φ(2), φ(1). Bài học này biến quan sát nhỏ đó thành một công cụ trả lời một triệu câu hỏi với n tới 10^7.

BÀI TOÁN. TỔNG GCD [TONGGCD]

Với số nguyên dương n, đặt P(n) = gcd(1, n) + gcd(2, n) + ... + gcd(n, n). Cho q truy vấn, mỗi truy vấn một số n, hãy in P(n).

Dữ liệu vào:

- Dòng đầu chứa q ($1 \le q \le 10^6$).
- q dòng sau, mỗi dòng một số nguyên n ($1 \le n \le 10^7$).

Kết quả:

- q dòng, mỗi dòng là P(n) tương ứng.

Dữ liệu vào	Kết quả	Giải thích
3 1 6 12	1 15 40	P(6) = 1 + 2 + 3 + 2 + 1 + 6 = 15. P(12) = 1 + 2 + 3 + 4 + 1 + 6 + 1 + 4 + 3 + 2 + 1 + 12 = 40.

1. Ý tưởng ngây thơ: Tính từng gcd

Với mỗi truy vấn, cộng gcd(i, n) cho i từ 1 tới n.

Lê Thành Nam

Code ý chính: Cách ngây thơ cho một truy vấn

```
long long total = 0;
for (int i = 1; i <= n; i++) total += __gcd(i, n);
```

CÁCH NGÂY THƠ CHẬM TỚI ĐÂU?

• Một truy vấn tốn n lần tính gcd, mỗi lần O(log n): khoảng $10^7 \cdot 23 \approx 2{,}3 \cdot 10^8$ phép tính với $n = 10^7$.

• Với $q = 10^6$ truy vấn: khoảng $2 \cdot 10^{14}$ phép tính, tức là vài ngày.

• Kể cả khi ta nhớ lại đáp án của các n đã gặp, đề vẫn có thể hỏi 10^6 giá trị n khác nhau.

BẾ TẮC

Mỗi truy vấn không được tốn quá vài chục phép tính. Cách duy nhất là **chuẩn bị sẵn** P(n) cho mọi $n \le 10^7$, mà muốn vậy thì cả bước chuẩn bị chỉ được phép tốn cỡ O(N) với $N = 10^7$. Ta cần hai thứ: một công thức tính P(n) từ các "mảnh nhỏ" của n, và một cách duyệt các số tới N sao cho mỗi số được xử lí đúng một lần.

Lê Thành Nam

2. Phá bế tắc, bước một: Nhóm theo giá trị của gcd

Bước 1 Đếm số i có gcd(i, n) = d

Giá trị gcd(i, n) luôn là một ước d của n. Ta đổi câu hỏi "mỗi i đóng góp bao nhiêu" thành "mỗi giá trị d xuất hiện bao nhiêu lần". Nếu gcd(i, n) = d thì $i = d \cdot j$ với $1 \le j \le n/d$, và $\gcd(j, \frac{n}{d}) = 1$. Ngược lại, mọi j như vậy cho một i có gcd(i, n) = d. Số các j này chính là φ(n/d), số các số trong $[1, \frac{n}{d}]$ nguyên tố cùng nhau với $\frac{n}{d}$.

NHÓM THEO GCD

$P(n) = \Sigma_{d \mid n}\, d \cdot \varphi(n/d)$

Bước 2 Chạy tay với n = 12

d	n/d	φ(n/d)	Các i có gcd = d	d · φ(n/d)
1	12	4	1, 5, 7, 11	4
2	6	2	2, 10	4
3	4	2	3, 9	6
4	3	2	4, 8	8
6	2	1	6	6
12	1	1	12	12

Tổng cột cuối: 4 + 4 + 6 + 8 + 6 + 12 = 40, khớp với ví dụ. Cột "số i" cũng cho thấy 12 số từ 1 tới 12 được chia đều vào các nhóm, mỗi số đúng một nhóm: 4 + 2 + 2 + 2 + 1 + 1 = 12.

Công thức này đã cho một lời giải $O(\sqrt{n})$ cho mỗi truy vấn: phân tích n, liệt kê ước, tính φ. Với $q = 10^6$ thì vẫn còn khoảng $3 \cdot 10^9$ phép tính. Ta cần đi xa hơn.

3. Phá bế tắc, bước hai: Hàm nhân tính

Một hàm số học f (xác định trên các số nguyên dương) gọi là **nhân tính** nếu f(1) = 1 và f(a · b) = f(a) · f(b) **mỗi khi gcd(a, b) = 1**. Nhiều hàm quen thuộc là nhân tính:

Hàm	Ý nghĩa	Giá trị tại lũy thừa nguyên tố p^k
φ(n)	số số trong [1, n] nguyên tố cùng nhau với n	$p^k - p^{k-1}$
d(n)	số ước của n	k + 1
σ(n)	tổng các ước của n	$1 + p + \ldots + p^k$
μ(n)	hàm Möbius (Bài 3)	−1 nếu k = 1; 0 nếu k ≥ 2

Khi $n = p_1^{e1} \cdot p_2^{e2} \cdots p_r^{er}$, hàm nhân tính được tính bằng tích $f(p_1^{e1}) \cdot f(p_2^{e2}) \cdots f(p_r^{er})$. Vậy **chỉ cần biết f tại các lũy thừa nguyên tố**.

VÌ SAO P LÀ HÀM NHÂN TÍNH?

Giả sử gcd(a, b) = 1. Mỗi ước d của ab tách duy nhất thành $d = d_1 \cdot d_2$ với $d_1 \mid a$, $d_2 \mid b$ ($d_1 = \gcd(d, a)$, $d_2 = \gcd(d, b)$). Khi đó $\frac{ab}{d} = (\frac{a}{d_1})(\frac{b}{d_2})$, hai thừa số nguyên tố cùng nhau nên $\varphi(\frac{ab}{d}) = \varphi(\frac{a}{d_1}) \cdot \varphi(\frac{b}{d_2})$. Do đó $P(ab) = \Sigma_{d1 \mid a} \Sigma_{d2 \mid b} d_1 d_2 \cdot \varphi(\frac{a}{d_1}) \cdot \varphi(\frac{b}{d_2}) = P(a) \cdot P(b)$.

Lập luận này đúng cho mọi tổng dạng $\Sigma_{d \mid n} f(d) \cdot g(n/d)$ với f, g nhân tính (gọi là **tích Dirichlet** của f và g): tích Dirichlet của hai hàm nhân tính là hàm nhân tính.

Tính P tại lũy thừa nguyên tố: các ước của p^k là p^j với $0 \le j \le k$. Số hạng j = k bằng $p^k \cdot \varphi(1) = p^k$. Với j < k, số hạng là $p^j \cdot \varphi(p^{k-j}) = p^j \cdot p^{k-j-1} \cdot (p - 1) = p^{k-1} \cdot (p - 1)$, có k số hạng như vậy.

GIÁ TRỊ CỦA P TẠI LŨY THỪA NGUYÊN TỐ

$P(p^k) = p^k + k \cdot p^{k-1} \cdot (p - 1) = (k + 1) \cdot p^k - k \cdot p^{k-1}$

Kiểm tra: P(4) = 3 · 4 − 2 · 2 = 8, đúng bằng gcd(1, 4) + gcd(2, 4) + gcd(3, 4) + gcd(4, 4) = 1 + 2 + 1 + 4. P(12) = P(4) · P(3) = 8 · 5 = 40.

4. Phá bế tắc, bước ba: Sàng tuyến tính

Sàng Eratosthenes gạch các bội của từng số nguyên tố, nên một hợp số bị gạch nhiều lần: 12 bị gạch bởi 2 và bởi 3, 30 bị gạch ba lần. Tổng số lần gạch là O(N log log N), vẫn nhanh,

Lê Thành Nam

nhưng việc "gạch nhiều lần" khiến ta không biết lúc nào nên **tính giá trị hàm** cho số đó. Sàng tuyến tính sửa điều này: mỗi hợp số x bị gạch đúng một lần, bởi cặp (p, i) với p = lp(x) là **thừa số nguyên tố nhỏ nhất** của x và $i = \frac{x}{p}$.

Bước 1 Thuật toán

Duyệt i = 2, 3, ..., N. Nếu i chưa bị gạch, i là số nguyên tố, thêm vào danh sách primes. Sau đó, duyệt các số nguyên tố p trong primes theo thứ tự tăng dần: gạch x = p · i (ghi lp(x) = p); **nếu p chia hết i thì dừng**.

Code ý chính: Sàng tuyến tính

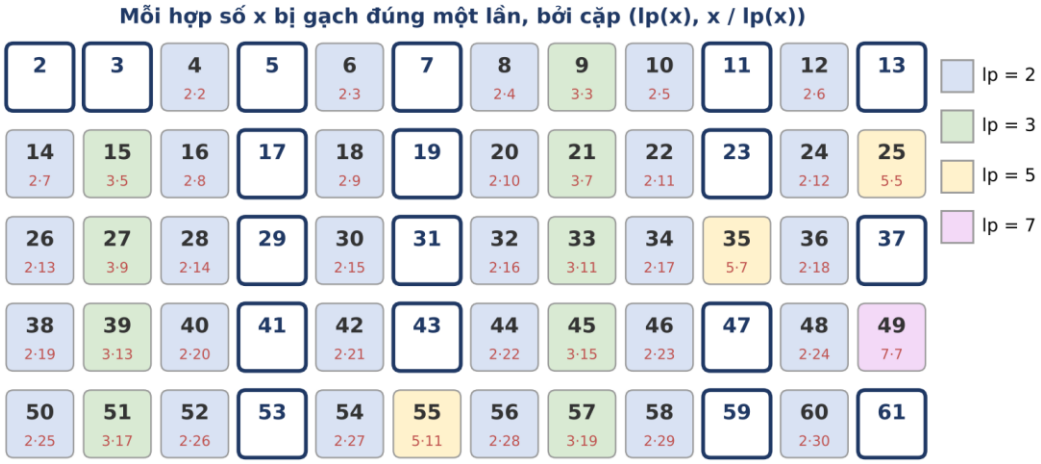
```
for (int i = 2; i <= N; i++) {
    if (lp[i] == 0) { lp[i] = i; primes.push_back(i); }
    for (int p : primes) {
        if ((long long)p * i > N) break;
        lp[p * i] = p;                // p = thua so nho nhat cua p*i
        if (i % p == 0) break;        // p lon hon se khong con la lp
    }
}
```

Bước 2 Chạy tay với N = 20

i	primes sau bước i	Số bị gạch	Dừng vì
2	2	4	2 chia hết 2
3	2, 3	6, 9	3 chia hết 3
4	2, 3	8	2 chia hết 4
5	2, 3, 5	10, 15	5 · 5 = 25 > 20
6	2, 3, 5	12	2 chia hết 6
7	2, 3, 5, 7	14	3 · 7 = 21 > 20
8	...	16	2 chia hết 8
9	...	18	3 · 9 = 27 > 20
10	...	20	2 chia hết 10

Mười một hợp số không vượt quá 20 (4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20) xuất hiện ở cột thứ ba, mỗi số đúng một lần. Lưu ý 12 bị gạch ở i = 6 (12 = 2 · 6) chứ không phải ở i = 4 (vì khi i = 4, vòng lặp dừng ngay sau p = 2).

Lê Thanh Nam



Hình 1.1. *Sàng tuyến tính tới 61: mỗi hợp số ghi cặp (lp(x), x/lp(x)) đã gạch nó, màu theo lp(x)*

Lê Thanh Nam

VÌ SAO MỖI HỢP SỐ BỊ GẠCH ĐÚNG MỘT LẦN?

Ít nhất một lần. Cho x là hợp số, p = lp(x), i = $\frac{x}{p}$. Mọi thừa số nguyên tố của i đều ≥ p, nên khi xử lí i, các số nguyên tố đứng trước p trong primes đều không chia hết i và vòng lặp không dừng trước p. Hơn nữa p ≤ i do p có trong primes. Vậy x = p · i được gạch.

Nhiều nhất một lần. Khi gạch x = p · i, ta luôn có p ≤ lp(i) (vì vòng lặp dừng ngay khi gặp p chia hết i), nên p = lp(i). Cặp (p, i) bị xác định duy nhất bởi x: p = lp(x), i = $\frac{x}{p}$.

SÀNG TUYẾN TÍNH TỐN BAO NHIÊU?

• Mỗi lần thực hiện thân vòng lặp trong ứng với một hợp số khác nhau (cộng thêm một lần `break` cho mỗi i): tổng cộng O(N).

• Với N = 10^7: khoảng 10^7 lần gạch, mất chừng 0,1 giây. Danh sách primes có 664 579 phần tử.

• Bộ nhớ: mảng lp kiểu int tốn 4N byte = 40 MB. Hãy cộng bộ nhớ của mọi mảng trước khi nộp bài.

5. Tính hàm nhân tính ngay trong sàng

Khi gạch x = p · i, có hai trường hợp:

- **p không chia hết i:** gcd(p, i) = 1 nên f(x) = f(p) · f(i).
- **p chia hết i:** p là lp của cả i lẫn x, nên ta cần biết "phần p" của i. Lưu thêm pk[i] = lũy thừa lớn nhất của lp(i) chia hết i và ex[i] là số mũ tương ứng. Khi đó x = (pk[i] · p) · ($\frac{i}{pk[i]}$),

hai thừa số nguyên tố cùng nhau, và f(x) = f(pk[i] · p) · f($\frac{i}{pk[i]}$). Giá trị f tại lũy thừa nguyên tố tính trực tiếp bằng công thức. Trường hợp thứ hai có công thức gọn hơn mà không cần pk:

Hàm	f(p)	f(p · i) khi p không chia hết i	f(p · i) khi p chia hết i (i chứa p^e)
φ	p − 1	φ(i) · (p − 1)	φ(i) · p
μ	−1	−μ(i)	0
d	2	2 · d(i)	d(i) / (e + 1) · (e + 2)
P	2p − 1	P(i) · (2p − 1)	P(i / p^e) · P(p^{e+1})

Dòng φ đúng vì φ(n) = n · Π(1 − $\frac{1}{q}$) trên các ước nguyên tố q của n: nhân thêm một thừa số p đã có sẵn thì tập ước nguyên tố không đổi, φ chỉ nhân thêm p. Dòng d cần biết số mũ e, nên ta lưu thêm mảng ex.

6. Chương trình hoàn chỉnh

Chương trình tính P(n) cho mọi n ≤ 10^7 bằng sàng tuyến tính, sau đó trả lời mỗi truy vấn trong O(1). P(n) ≤ n · d(n) ≤ 10^7 · 448 nên phải dùng long long; mảng P tốn 80 MB, pk thêm 40 MB, ex (kiểu unsigned char) thêm 10 MB, tổng khoảng 130 MB.

TONGGCD.CPP: Sàng tuyến tính tính hàm nhân tính P, O(N + q)

```
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;
typedef long long ll;
const int MAXN = 10000000;

int main() {
    if (freopen("TONGGCD.INP", "r")) {
        freopen("TONGGCD.INP", "r", stdin);
        freopen("TONGGCD.OUT", "w", stdout);
    }
    // P(n) = tong gcd(i, n), 1 <= i <= n; P nhan tinh va
    // P(p^k) = (k + 1) * p^k - k * p^(k-1)
    // pk[i] = luy thua cua thua so nguyen to nho nhat trong i
    // ex[i] = so mu tuong ung
    vector<ll> P(MAXN + 1, 0);
    vector<int> pk(MAXN + 1, 0), primes;
    vector<unsigned char> ex(MAXN + 1, 0);
    P[1] = 1;
    for (int i = 2; i <= MAXN; i++) {
        if (pk[i] == 0) {                          // i nguyen to
            pk[i] = i; ex[i] = 1;
            P[i] = 2LL * i - 1;
            primes.push_back(i);
        }
        for (int p : primes) {
            if ((ll)p * i > MAXN) break;
            int x = p * i;
            if (i % p == 0) {                      // p chia het i
                pk[x] = pk[i] * p;
                ex[x] = ex[i] + 1;
                ll q = pk[x], k = ex[x];
```

Lê Thành Nam

```
                ll Pq = (k + 1) * q - k * (q / p);  // P(p^k)
                int rest = i / pk[i]; // phan nguyen to cung nhau voi p
                P[x] = P[rest] * Pq;
                break;
            }
            pk[x] = p; ex[x] = 1;
            P[x] = P[i] * P[p];                 // gcd(i, p) = 1
        }
    }
    int q;
    scanf("%d", &q);
    while (q--) {
        int n;
        scanf("%d", &n);
        printf("%lld\n", P[n]);
    }
    return 0;
}
```

Vài chi tiết đáng chú ý:

• Tích `(ll)p * i` được ép sang long long trước khi so sánh với MAXN; nếu để int, p·i có thể tràn khi i gần 10^7.

• Mảng pk vừa lưu lũy thừa của lp, vừa đóng vai trò đánh dấu: pk[i] = 0 nghĩa là i chưa bị gạch, tức i là số nguyên tố.

• Biến rest = $\frac{i}{pk[i]}$ có thể bằng 1 (khi i là lũy thừa nguyên tố); P[1] = 1 nên công thức vẫn đúng mà không cần câu lệnh if.

• Đọc 10^6 số bằng scanf thay vì cin không đồng bộ để không mất thời gian nhập xuất.

7. Mở rộng

Mở rộng 1: Tổng trên cả bảng. $\Sigma_{i \le n} \Sigma_{j \le n}$ gcd(i, j) cũng nhóm được theo giá trị của gcd, nhưng lần này dùng hằng thức gcd(i, j) = $\Sigma_{d \mid \gcd(i,j)}$ φ(d): mỗi d đóng góp φ(d) · ⌊n/d⌋^{2}. Đây là bài 1.2 ở Tập 2.

Mở rộng 2: Tổng các bội chung nhỏ nhất. $\Sigma_{i \le n}$ lcm(i, n) dẫn tới hàm g(n) = $\Sigma_{d \mid n}$ d · φ(d), cũng nhân tính. Bài 1.3 ở Tập 2.

Mở rộng 3: Bình phương số ước. d(n)^2 là hàm nhân tính với giá trị (k + 1)^2 tại p^k. Cập nhật trong sàng cần số mũ e của lp: bài 1.1.

Mở rộng 4: Khi nào không cần sàng tuyến tính? Nếu chỉ cần φ cho mọi số tới N, cách "sàng φ" kiểu Eratosthenes (với mỗi số nguyên tố p, nhân phi[m] với (1 − 1/p) cho mọi bội m của p) tốn O(N log log N), cũng đủ nhanh. Sàng tuyến tính thực sự cần khi hàm cần biết lp và số mũ của nó, hoặc khi N lớn tới mức hằng số log log N trở nên đáng kể.

Lê Thành Nam

BẪY THƯỜNG GẶP

• **Quên `break` khi p chia hết i:** sàng vẫn đánh dấu đúng số nguyên tố, nhưng một hợp số bị gạch nhiều lần và giá trị hàm bị ghi đè bằng công thức sai.

• **Dùng công thức f(p · i) = f(p) · f(i) cho mọi trường hợp:** đúng với φ khi p không chia hết i, sai khi p chia hết i (φ(4) = 2 chứ không phải φ(2) · φ(2) = 1).

• **Tràn int ở p · i:** phải so sánh `(long long)p * i > N`.

• **Quá bộ nhớ:** ba mảng long long cỡ 10^7 đã là 240 MB. Hãy chọn kiểu nhỏ nhất đủ chứa (unsigned char cho số mũ, int cho lp và pk).

• **Nhập xuất chậm:** 10^6 dòng in bằng `cout << endl` có thể mất hơn một giây vì endl xả bộ đệm mỗi lần.

GHI NHỚ

• Tổng theo gcd: nhóm theo giá trị d của gcd, đếm số phần tử trong mỗi nhóm bằng φ.

• Hàm nhân tính được xác định bởi giá trị tại các lũy thừa nguyên tố. Tích Dirichlet của hai hàm nhân tính là hàm nhân tính.

• Sàng tuyến tính gạch mỗi hợp số đúng một lần bởi lp của nó, cho phép tính mọi hàm nhân tính tới N trong O(N).

• Khi p chia hết i, cần tách phần lũy thừa của p (mảng pk, ex) để áp dụng tính nhân tính.

Lê Thành Nam

LUYỆN TẬP

Bài	Tên	Gợi ý liên hệ
1.1	BÌNH PHƯƠNG SỐ ƯỚC	Tính Σ $d(i)^2$ với i tới $2 \cdot 10^7$. Hàm d^2 nhân tính, cần số mũ của lp.
1.2	TỔNG GCD TOÀN BẢNG	Tổng gcd(i, j) trên bảng n × n. Viết gcd thành tổng φ trên các ước chung.
1.3	TỔNG BCNN	Tổng lcm(i, n) cho 10^6 truy vấn. Ghép cặp i với n − i rồi nhóm theo gcd.

Tự kiểm tra nhanh

1. Tính P(9) bằng công thức và bằng định nghĩa.

(Đáp số: $3 \cdot 9 - 2 \cdot 3 = 21$; định nghĩa: 1 + 1 + 3 + 1 + 1 + 3 + 1 + 1 + 9 = 21.)

2. Trong sàng tuyến tính, số 60 bị gạch ở bước i nào, với số nguyên tố p nào?

(Đáp số: i = 30, p = 2.)

3. Với $N = 10^7$, ba mảng: lp kiểu int, phi kiểu int, mu kiểu signed char tốn bao nhiêu bộ nhớ?
(Đáp số: 40 + 40 + 10 = 90 MB.)